

Location Aware Code Offloading on Mobile Cloud with QoS Constraint

Samia Tasnim, Mohammad Aatur Rahman Chowdhury, Kishwar Ahmed, Niki Pissinou, and S. S. Iyengar
 School of Computing and Information Sciences
 Florida International University
 Miami, Florida 33199

Email: {stasn002, mchow017, kahme006, pissinou}@fiu.edu, iyengar@cis.fiu.edu

Abstract—Mobile applications can be enhanced to a great extent by using the offloading mechanism in an energy efficient manner to the bounty resourceful clouds. Due to the huge demand of smart phones, the issue of providing more processing capability to this resource constraint device is getting more concern now-a-days. In this paper, a method level offloading mechanism has been proposed where no prior image of the mobile device is needed to be transferred to the cloud. The application is partitioned at different points where the migration of the execution thread is performed from mobile device to nearby resourceful cloud to get the best execution performance in optimal energy cost. The mobile can complete the execution after the partitioned thread returns back from the cloud to the device. This mechanism increases scalability as well as performance in the form of faster execution speed of the mobile devices. Moreover, we consider the mobility of the mobile device and propose a solution to find the best cloud instance on the move. To find out which cloud to offload, the communication latency, capacity, and current load at individual clouds are considered to find out the best cloud to offload to ensure better service for the mobile device. The proposed solution has been simulated and compared against CloneCloud in two different simulation scenarios where we show that our method performs superior to CloneCloud.

I. INTRODUCTION

Despite significant improvement in mobile technologies, the capacity of mobile devices are inadequate to satiate their hunger for resources (e.g., energy, CPU etc). With the increase of network resources namely high performance clouds, it is constantly investigated to find the optimal partitioning between the loads of mobile device and the cloud. Much of the recent research has been focused on different aspects of offloading. Along with offloading, the location of the servers used for computational benefits is also important. In many cases the transfer latency adversely affects the performance of the offloading mechanism. We investigate that, the communication latency between the device and the cloud, and the current service load of the cloud is a factor in making offloading decision. CloneCloud[1] provides a solution of code offloading by using per device clone in the cloud. MAUI[2] also provides a similar solution. CloneCloud divides the code into partitions and uses static analysis and dynamic profiling to build decision trees to make the offloading decision. We observed that, in this way, it could take a significant number of runs to gather

sufficient information to make an optimal choice. It also incurs the cost of sending the codes to the cloud for profiling, which may be significant in low bandwidth scenarios.

In our work, we focus on the network latency and relative distance between the device and the service provider. Limiting the latency of communication is of main importance. We also consider the mobility of the mobile device, so that the device can communicate with the nearest possible server in terms of cost. We propose a solution to use the existing network topology to keep track of the nearest cloud servers in a particular location and use the existing mechanisms (e.g., hand-off) to trigger our proposed location based server selection algorithm. We take three steps approach to reduce this cost and keep this minimized throughout the service period. First, we suggest multiple servers in strategically placed locations to effectively minimize the latency of device-cloud communication. Later, we explain how we can use the existing network topology to select and maintain an active connection with the server. Second, we propose platform independent service at the cloud, which will increase the number of services, and provide privacy, as we do not keep any prior image of the mobile device. Third, we propose to build cost tree for different partitions of an application by comparing the current server capacity vector (SCV), with the mobile capacity vector (MCV), which results in reduced cost in terms of data transfer as well as immediate decision making. The ratio of SCV and MCV has been termed as gain factor. SCV denotes a set of constraints (e.g., CPU speed, memory, energy consumption etc) which determines the performance capability of the cloud server while MCV denotes a similar set of constraints for the mobile device. We simulated our proposed solution and compared it with the performance of CloneCloud. The comparison results show that our LocationAware offloading method performs better than CloneCloud in terms of throughput, delay and energy consumption.

We organize our paper in the following manner. In section 2, we discuss our work with related contemporary work. In section 3, we discuss different parts of our system and define an algorithm for offloading. In section 4, we discuss the simulation setup while in section 5, the performance evaluation

is discussed. Finally, concluding remarks are offered in section 6.

II. RELATED WORK

In this section, we discuss the works that are most pertinent to our work. Mobile image offloading to the cloud server depends on some QoS (e.g., energy, delay etc) constraints along with some other persistent issues (e.g., network connectivity, location management etc). [3] considered intermittent connectivity of mobile devices where the mobile device uses the resource of other devices which are located near to it over a Wi-Fi network. The authors of [4] considered computation offloading among a set of collaborative mobile devices named as Mobile Device Cloud. Their main aim is to boost the battery life of the collaborating mobile devices. There has been some good amount of work on privacy issues on offloading to cloud server (e.g., [5], [6], [7]). Moreover, there is another significant line of work where authors have discussed efficient way of detecting malware of the mobile devices on the cloud and thus decreases the computational overhead that may incur if done on the mobile device itself. In [6], record and replay method has been used. In [8], the installed app is uploaded from mobile device to the cloud server where the scanning is performed using third party tools like: Kaspersky, VirusChief etc. Uploading apps from mobile device and scanning them takes significant time, which results in good amount of delay for the user to get the result. Thus, the benefit of cloud cannot be achieved in the full phase.

The decision making process of whether to offload the service to the cloud or not depends on some QoS constraints and also power usage [9], [10]. The paper [11] discusses the decision issue of whether to offload or not depending on bandwidth issue where they have considered two types of clone on the cloud: off-clone and back-clone. The authors have shown a comparison among the costs that were incurred to keep the respective clone synch with the mobile device. They also considered the communication overhead that is necessary to achieve different levels of synchronization. In ThinkAir [12], device specific clone is not needed. They used a single server to accommodate all the requests. However, Sonora [13] supports execution partitioning by introducing a novel failure recovery protocol to put more emphasis on fault tolerance.

There are different ways of offloading discussed in past literature. For example, in [6], the whole image of smart phone is transferred to the cloud, while according to [1], part of smart phone image is transferred keeping the rest of the part inside the mobile for further computation. In [2], the code is offloaded to the cloud which is another significant part of literature representing image offload to the cloud. An extension of CloneCloud is [14], which has improved on the intermittent connectivity issue of CloneCloud. Moreover, this paper has proposed a solution on the decision making process of how much of a whole service to be transferred to the cloud depending on the

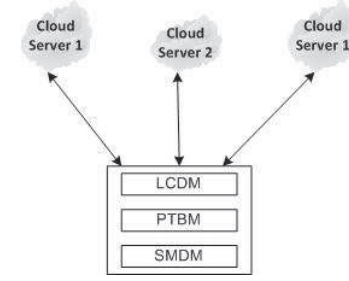


Fig. 1. Overall Architecture Design

tradeoff of completion time and service migration time between clouds. Another line of work related to taking decision on distribution of cloud geographically is [15], where the authors have proposed a solution on service distribution among different clouds, focusing mainly on time critical events. The authors of [16] used component based application partitioning technique. They have put more emphasis on the resource sharing in the cloud servers by finding out the kinship between varied jobs at the cloud servers.

However, to the best of our knowledge, no work has been done on offloading of service code considering location change of the mobile device, and also depending on the partitioning of the services.

III. ARCHITECTURE

In our system, we propose to use application independent clone servers which are located in geographically distributed manner. Clone server distribution could also be designed according to the client request load. By application independent, we mean that no prior per device image is needed to be stored in the server. Request from the mobile device will be served independent of the configuration of the mobile device.

To incorporate location change of the mobile device, before offloading, the cloud server which demands less cost in terms of communication latency and possible execution time is chosen. Depending on the current location of the user, the service execution platform may need to change.

A. Overall Design

The overall architecture design of our proposed system is shown in Fig. 1. In the following we discuss the main components of the overall system:

1) *Location Change Decision Module (LCDM)*: Whenever there is change of location of a Mobile Device (MD) in terms of location area, this module helps to take decision on whether to continue using the previous cloud server for service execution or to assign a new cloud server for the new location. For example, whenever a MD goes away from a current location area to a new location area, executing the service at a server located closest to that location would be good choice. However, the mobile device should have knowledge of the nearest cloud

Algorithm 1 Update Native CloudList

```

1: if  $List_n == List_c$  then
2:   No calculation required
3: else if  $List_n \subset List_c$  then
4:    $S = List_c - List_n$ 
5:   for all  $S_i \in S$  do
6:     Calculate gain factor for  $S_i$ 
7:   end for
8:   CreateProfileTree()
9: else if  $List_c \subset List_n$  then
10:   $List_n = List_c$ 
11: else
12:  for all  $S_i \in List_c$  do
13:    Calculate gain factor for  $S_i$ 
14:  end for
15:   $List_n = List_c$ 
16:  CreateProfileTree()
17: end if

```

servers to make decision of changing the active server. In this paper, we propose to use the existing network topology to provide the MD with the list of nearest cloud servers. We suggest that, VLR of the location area will keep a list of nearest cloud servers. We propose an algorithm to keep a list of possible servers to offload in the MD. The hand-off mechanism of current 3G networks will work as a trigger to our algorithm and send the list of nearest cloud servers to the MD.

2) *Profile Tree Building Module (PTBM)*: This module, which is a part of LCDM, takes as input the SCV 's of the cloud servers from the native list. It calculates the new profile tree depending on the current status of the application, where current status denotes which nodes are still in the server for computation and which nodes have already been served.

3) *Service Migration Decision Module (SMDM)*: This module is responsible for taking decision on whether to transfer the service to the cloud based on service profile tree. While considering latency as decision criteria, we take into consideration communication latency along with service latency on the cloud.

B. Updating the neighboring cloud server list

The module LCDM is responsible for this update action. First, we define the following key terms which we use in Algorithm 1.

- Active cloud server (A_{cl}): The cloud server currently serving the MD's requests.
- Native list of neighboring cloud servers ($List_n$): Current list maintained in the MD for nearest cloud servers.
- Current list of neighboring cloud servers ($List_c$): List provided by the VLR when a hand-off is initiated.

Now we briefly explain the algorithm: when $List_c$ is sent from the network, it is checked whether A_{cl} is present in the list or not. In case A_{cl} is found in the list, $List_c$ is compared

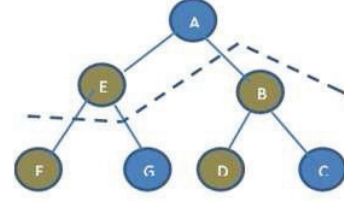


Fig. 2. Profile Tree

with $List_n$, and no new calculation is needed when the lists are similar. If $List_c$ is a subset of $List_n$, the absent cloud servers in $List_c$ are removed from $List_n$. While in case of $List_n$ being a subset of $List_c$, gain factors are calculated for all the new cloud servers in $List_c$. If none of the servers in $List_n$ is present in the $List_c$ then $List_n$ is replaced by $List_c$.

C. Building the profile tree

The module PTBM is responsible for building the profile tree. To build the execution trace for cloud, instead of executing all the methods there, we perform the following steps:

- Collect the current SCV from the nearby cloud servers. This is a single step process which only requires the server to send its SCV to the client until the next notification time, which depends on the server load change. If there is a drastic change in the server load, immediate notification about the current SCV is sent to the client. Otherwise, the SCV update is provided to the client in periodic manner. Server sends the SCV to the mobile device, only if the device has some outstanding task in the server at that time, e.g., the device is still connected for service. Sending the SCV when server load changes drastically helps the mobile device to build a new profile tree to make optimal decision regarding choice of the server.
- Use the MCV to compute gain factor which insinuates the overall performance improvement achieved due to offloading.
- The profile tree for the cloud servers will be estimated simply by multiplying the gain factors to the native profile tree.

D. Identifying the nodes and cloud server for migration

We partition the service code into N parts, denoting each part of the service code by S_i , where $i \in 1, 2, \dots, N$. These partitions are actually methods of the executable similar to [1]. Now to dynamically transfer a part of a service, we take into consideration the cost of migration (m_i) along with the service execution cost (E_i), where E_i can be found from the profile tree built for each server. Moreover, we also take decision based on QoS constraint (energy, latency etc). To select the platform, either mobile device or cloud for service code execution at optimal cost value, the method presented in [9], [10] has been used. The SMDM module is responsible to perform this action.

E. Connecting to the new cloud server

After the server selection is done, connecting to the new cloud server and disconnecting the previous one is not a straight

forward choice. Some of the nodes in the profile tree may still be executing in the previous one, when the decision to connect to new cloud server has been made. We consider the following options for smooth transition between the servers:

- Connection to the old cloud server will be uncoupled, and the nodes which were being serviced at this time will be sent to the new cloud server for computational benefit. The disadvantage of this scheme is that a lot of executions have to be repeated if multiple nodes were being serviced at the cloud at that moment.

- Connection to the existing server will be kept until the current serving nodes are returned to the *MD*. Consequently, the overhead of executing the same node multiple time is removed. However, the overhead of increased transmission delay remains in effect.

- Connection to the old server will be closed immediately, but address of the new server will be sent to the currently executing one before the connection is closed. The execution will continue in the disconnected server and will be sent to the new server after completion. Therefore, the *MD* does not have to keep two active connections at the same time.

As an example, we consider the profile tree (Fig. 2) where nodes *A*, *C* and *G* are executed on the *MD* and others are executed on the cloud. At this instance of the tree, node *A* and *E* are complete. But at the time of connecting to new cloud server, node *B* is executing in the old cloud server. Hence, we can disconnect the connection with old server immediately and send node *B* to the new server (option 1), or keep the existing connection active until node *B*'s execution is complete (option 2). Otherwise, disconnect the connection between *MD* and old cloud server. The new cloud server receives the result of node *B*'s execution from the old one and send it back to *MD* (option 3). Choice of option among the above three depends on the size and nature of the nodes. In the simulation, we have kept two connections active at the same time (option 2) to reduce overhead of multiple execution (option 1) and overhead of cloud-to-cloud connection establishment (option 3).

IV. SIMULATION SETUP

In this section, we describe the environment we created and different parameters we used for comparison with the algorithm presented in CloneCloud. Our simulation setup was built and tested using simulation software ns-2. The area under consideration was a $670m \times 670m$ square with one mobile node and two base stations, each having coverage of 75 meter.

We considered two scenarios for simulation where each of them had multiple cloud servers, one base station for each cell area and mobile device moving from one cell area to another causing hand-off to take place. Our algorithm uses cloud servers positioned at different locations for running application depending on the workload requirement of that application. Whenever hand-off takes place for a mobile device, we recalculate the distance between it and the cloud server,

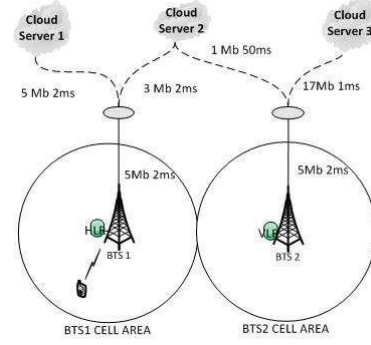


Fig. 3. Simulation Scenario for set up 1

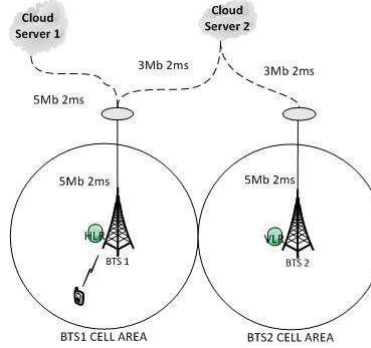


Fig. 4. Simulation Scenario for set up 2

and based on the distance, find the nearest cloud server with required capacity to execute the application. Instead of sending the whole code to the cloud server, however, we send a partial code to the server and thus optimize the execution of code at cloud server or mobile device.

In both of the topologies (Fig. 3, Fig. 4), the movement of *MD* begins in 100 second, while after 200 second, it returns towards the starting position and the simulation stops at 250 second. The simulation has been performed for both our proposed solution (LocationAware) and for CloneCloud.

V. PERFORMANCE EVALUATION

This section outlines the comparison of our algorithm with other algorithm (CloneCloud) based on the simulation setup we have used. We compare on the basis of the following comparison criteria:

- Throughput: This is a measurement of successful message transfer rate between a mobile device and cloud server. We calculate the number of bytes transferred per second for both of the scenarios we created and for both our algorithm and the algorithm used in CloneCloud.

In the simulation scenario of Fig. 3, earlier, code offloading was performed to Cloud Server 1. Later, when the *MD* moved to cell area 2, the current neighboring list was updated by Cloud Server 2 and Cloud Server 3. The load in Cloud Server 3

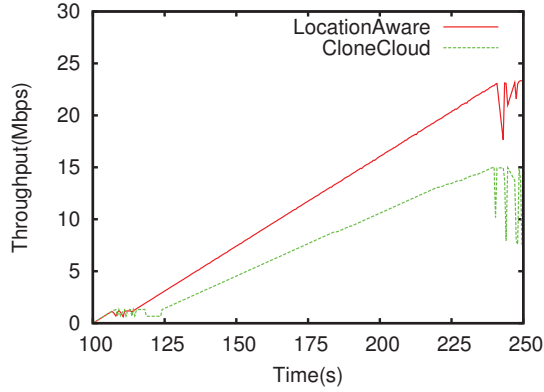


Fig. 5. Throughput Comparison between CloneCloud and LocationAware Code offloading for topology 1

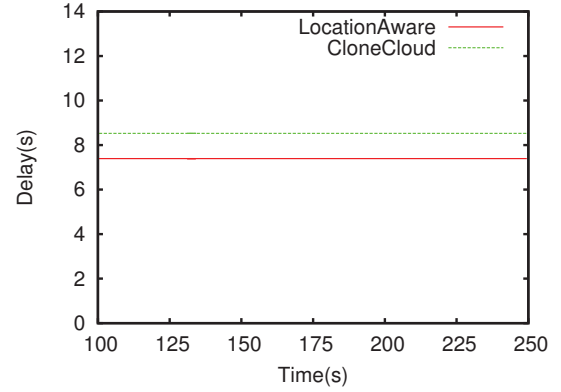


Fig. 8. Delay Comparison between CloneCloud and LocationAware Code offloading for topology 2

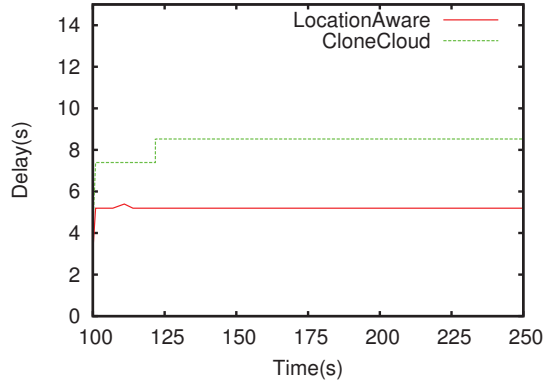


Fig. 6. Delay Comparison between CloneCloud and LocationAware Code offloading for topology 1

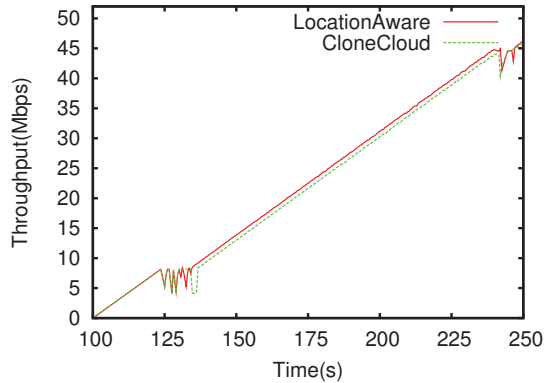


Fig. 7. Throughput Comparison between CloneCloud and LocationAware Code offloading for topology 2

was low while the channel bandwidth towards Cloud Server 3 was three times better than the link towards Cloud Server 1. It was nearer to *MD* in comparison to Cloud Server 1 too. During the transition from cell area 1 to cell area 2, the selected

cloud server for offloading was changed from Cloud Server 1 to Cloud Server 3 to ensure better throughput. Even though some extra time is required for performing handover, the resultant throughput surpasses this loss.

On the other hand, in CloneCloud, throughout the whole execution period, the code offloading was performed between *MD* and Cloud Server 1 which contains the prior image. In the simulation set up of Fig. 4, the set of neighboring cloud server was same for both the cell areas. The load in comparison to the capacity at Cloud Server 1 was greater than Cloud Server 2 which influenced the LocationAware offloading method to choose Cloud Server 2 to offload code. Fig. 5 and Fig. 7 demonstrate the better throughput gained (approximately 50%) by our LocationAware algorithm in comparison to CloneCloud for both of the simulation scenarios.

- **Delay:** We measure the delay by calculating the average time it takes for a packet to reach the destination and return to the source. Whenever there is handoff for a mobile host, the delay varies depending on the position of the cloud server where the code is to be executed. In Fig. 6 and Fig. 8, it can be seen that for our algorithm, the overall delay in the communication between the *MD* and offloaded cloud is less (approximately 60%) in comparison to CloneCloud.

- **Power:** Another QoS constraint that we take as a base of comparison is power consumption of mobile device when they move around different cell areas. Depending on the distance of the cloud server to offload from the cell area on which the mobile device is currently positioned, power consumption by that mobile device changes.

The battery status of the mobile device during the simulation has been measured. The battery model [17] for the mobile node has been used for this purpose. We have used parameter values which are estimated for lithium-ion cells [18]. Fig. 9 shows that although both the algorithms, LocationAware and CloneCloud ran for equal execution time, the overall throughput achieved for CloneCloud was much less than our

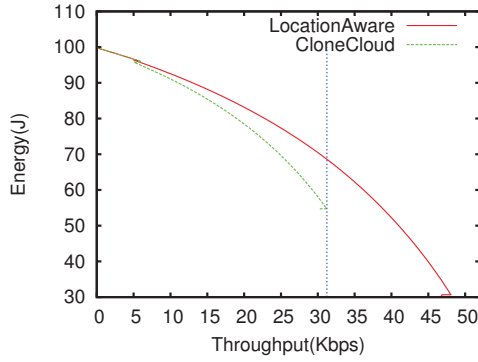


Fig. 9. Remaining Energy Comparison between CloneCloud and LocationAware Code offloading for topology 1

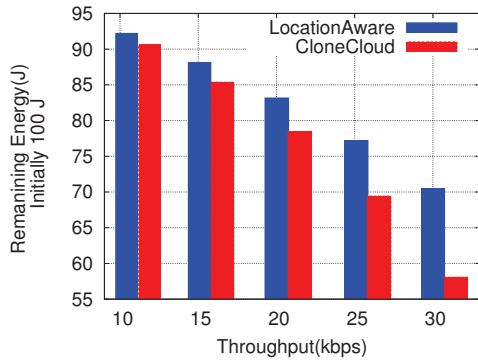


Fig. 10. Remaining energy comparison for achieving same throughput

LocationAware algorithm. Fig. 10 demonstrates the comparison of remaining energy in the mobile device for different values of throughput. It is evident from Fig. 10 that LocationAware algorithm incurs less energy in comparison to CloneCloud to provide same amount of service.

VI. CONCLUSION

Our proposed system incorporates the location change of the mobile device before code offloading to a cloud server. Moreover, we avoid the extra cost of dynamic profiling incurred while executing the process in the cloud by introducing the concept of gain factor to estimate the execution time in the cloud. Instead of a fixed cloud server, depending on the geolocation of the mobile device, as well as the existing load and capacity of the nearby cloud servers, the choice of the cloud server is made for code offload. The simulation results show how our LocationAware algorithm outperforms the CloneCloud algorithm in perspective of throughput, delay and energy consumption.

ACKNOWLEDGMENT

We would like to thank our colleague Md. Shakil Ahamed for his valuable contribution to the project.

REFERENCES

- [1] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, and A. Patti, "Clonecloud: elastic execution between mobile device and cloud," in *Proceedings of the sixth conference on Computer systems*, ser. EuroSys '11, 2011.
- [2] E. Cuervo, A. Balasubramanian, D.-k. Cho, A. Wolman, S. Saroiu, R. Chandra, and P. Bahl, "Maui: making smartphones last longer with code offload," in *Proceedings of the 8th international conference on Mobile systems, applications, and services*, 2010.
- [3] C. Shi, V. Lakafosis, M. H. Ammar, and E. W. Zegura, "Serendipity: enabling remote computing among intermittently connected mobile devices," in *Proceedings of the thirteenth ACM international symposium on Mobile Ad Hoc Networking and Computing*, ser. MobiHoc '12, 2012.
- [4] A. Mtibaa, A. Fahim, K. A. Harras, and M. H. Ammar, "Towards resource sharing in mobile device clouds: power balancing across mobile devices," in *Proceedings of the second ACM SIGCOMM workshop on Mobile cloud computing*, 2013.
- [5] Z. Zhou and D. Huang, "Efficient and secure data storage operations for mobile cloud computing," in *CNSM*, 2012.
- [6] G. Portokalidis, P. Homburg, K. Anagnostakis, and H. Bos, "Paranoid android: versatile protection for smartphones," in *Proceedings of the 26th Annual Computer Security Applications Conference*, ser. ACSAC '10, 2010.
- [7] S. Zonouz, A. Houmansadr, R. Berthier, N. Borisov, and W. Sanders, "Seccloud: A cloud-based comprehensive and lightweight security solution for smartphones," *Computers & Security*, 2013.
- [8] C. Jarabek, D. Barrera, and J. Aycock, "Thinav: truly lightweight mobile cloud-based anti-malware," in *Proceedings of the 28th Annual Computer Security Applications Conference*, 2012.
- [9] Y. Ye, N. Jain, L. Xia, S. Joshi, I.-L. Yen, F. Bastani, K. L. Cureton, and M. K. Bowler, "A framework for qos and power management in a service cloud environment with mobile devices," in *Service Oriented System Engineering (SOSE), 2010 Fifth IEEE International Symposium on*, 2010.
- [10] Y. Ye, L. Xiao, I.-L. Yen, and F. Bastani, "Leveraging service clouds for power and qos management for mobile devices," in *Cloud Computing (CLOUD), 2011 IEEE International Conference on*, 2011.
- [11] M. V. Barbera, S. Kosta, A. Mei, and J. Stefa, "To offload or not to offload? the bandwidth and energy costs of mobile cloud computing."
- [12] S. Kosta, A. Aucinas, P. Hui, R. Mortier, and X. Zhang, "Thinkair: Dynamic resource allocation and parallel execution in the cloud for mobile code offloading," in *INFOCOM*, 2012.
- [13] F. Yang, Z. Qian, X. Chen, I. Beschastnikh, L. Zhuang, L. Zhou, and J. Shen, "Sonora: A platform for continuous mobile-cloud computing," Technical Report. Microsoft Research Asia, Tech. Rep.
- [14] C. Shi, M. H. Ammar, E. W. Zegura, and M. Naik, "Computing in cirrus clouds: the challenge of intermittent connectivity," in *Proceedings of the first edition of the MCC workshop on Mobile cloud computing*, 2012.
- [15] C.-J. Wu, J.-M. Ho, and M.-S. Chen, "Time-critical event dissemination in geographically distributed clouds," in *Computer Communications Workshops (INFOCOM WKSHPS), 2011 IEEE Conference on*, 2011.
- [16] C. Mei, D. Taylor, C. Wang, A. Chandra, and J. Weissman, "Sharing-aware cloud-based mobile outsourcing," in *Proceedings of the 2012 IEEE Fifth International Conference on Cloud Computing*, 2012.
- [17] M. Handy and D. Timmermann, "Simulation of mobile wireless networks with accurate modelling of non-linear battery effects."
- [18] D. Rakhmatov, S. Vrudhula, and D. A. Wallach, "Battery lifetime prediction for energy-aware computing," in *Proceedings of the 2002 international symposium on Low power electronics and design*, 2002.